Übungsblatt 9

Java Web Stack, Maven & Web-Architekturmuster

5430UE Web and Data Engineering

17. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundlegendes Verständnis des Java Web Stacks und des Build-Managements mit Maven.
- Einblick in die serverseitige Implementierung von REST-Schnittstellen mit Spring Boot.
- Grundlegendes Verständnis von Web-Architekturmustern und Session-Tracking-Methoden.



Java Web Stack Begriffe



Java Web Stack Begriffe (1/2)

a) Gegeben sind folgende Begriffe:
View Template Engine, Application Server, Build Management Tool, Test Framework, CI/CD Tool, Datenbank, Web Framework

Erläutern Sie diese jeweils kurz in einem Satz.



Java Web Stack Begriffe (2/2)

b) Ordnen Sie die folgenden Technologien der richtigen Kategorie (Begriffe aus a)) im Java Web Stack zu und geben Sie eine kurze Beschreibung ihrer Aufgabe:

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	?	?
Tomcat	?	?
Maven	?	?
JUnit	?	?
Jenkins	?	?
H2	?	?
Spring Framework	?	?

Maven Grundlagen



Maven Grundlagen (1/2)

Machen Sie sich auf maven.apache.org/what-is-maven.html mit den Grundideen von Apache Maven vertraut.

1. Was ist Maven grundlegend und welche Kernprobleme der Softwareentwicklung löst es?
2. Was ist ein Maven-Artefakt? Wie ist eine Maven-Koordinate aufgebaut? Geben Sie ein konkretes Beispiel und erklären Sie die einzelnen Bestandteile.
3. Was bedeutet semantische Versionierung? Erklären Sie `MAJOR.MINOR.PATCH` anhand eines Beispiels.
4. Was versteht man unter einem Build-Lebenszyklus in Maven und warum ist dieses Konzept für Entwickler nützlich? Nennen Sie zudem mindestens vier Phasen des *Maven Default Lifecycle* in der richtigen Reihenfolge.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

Wer mit Spring Boot in einer modernen Entwicklungsumgebung wie *IntelliJ IDEA* arbeitet, kennt das: Man schreibt seinen Quellcode, klickt oben rechts auf das grüne Dreieck (Run-Button) und Sekunden später läuft das Backend. Von einem Werkzeug namens „Maven“ sieht man im Alltag auf den ersten Blick fast nichts.

Warum beschäftigen wir uns also mit Maven-Koordinaten und Build-Lebenszyklen? Die Antwort ist einfach: **Maven ist der unsichtbare Architekt Ihres Projekts.**

1. **Das „grüne Dreieck“ ist eine Abkürzung**(1/2) Wenn Sie in IntelliJ auf den Start-Button klicken, passiert im Hintergrund Folgendes:
   - Die IDE schaut in die Datei `pom.xml`, um zu prüfen, welche Java-Version und welche externen Bibliotheken (z. B. *Spring Web*, *Spring Data JPA*) Sie für Ihren Code angefordert haben.
   - IntelliJ nutzt im Hintergrund die Logik der Maven-Phase `compile`, um Ihre geschriebenen `.java`-Dateien in ausführbaren Bytecode (`.class`) zu übersetzen.
   - Danach startet die IDE die Anwendung sofort direkt aus diesen losen, frisch übersetzten Dateien.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

1. **Das „grüne Dreieck“ ist eine Abkürzung**(2/2) Das ist extrem praktisch, weil es beim Programmieren viel Zeit spart. Es führt jedoch oft zu dem Trugschluss, dass man Maven erst ganz am Ende für das spätere „Deployment“ (den Serverbetrieb) benötigt. In Wahrheit nutzt IntelliJ die von Maven definierte Struktur in jeder Sekunde der Entwicklung.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

2. **Der Lifecycle: Ihr Sicherheitsgurt für die Praxis**

   Während des Programmierens interessiert uns tatsächlich meistens nur die Phase `compile`. Doch der eigentliche Maven-Lebenszyklus (*Build Lifecycle*) ist eine feste Kette von Schritten, die strikt aufeinander aufbauen. Und diese Kette wird genau dann lebenswichtig, wenn wir die schützende und automatisierte Umgebung unserer IDE verlassen.

   Sobald ein Projekt fertiggestellt, im Team geteilt oder in eine CI/CD-Pipeline zur Server-Bereitstellung übergeben wird, entfaltet Maven seine volle Kontrollfunktion und erzwingt den Standard-Lebenszyklus von vorne bis hinten:

   - `validate` & `compile`: Der Quellcode wird formal auf syntaktische Korrektheit geprüft und komplett frisch übersetzt.
   - `test`: Maven sucht nach *allen* im Projekt hinterlegten automatisierten Tests (z. B. JUnit) und führt diese aus. Schlägt auch nur ein einziger Test fehl, bricht Maven den Prozess sofort ab. Dies ist ein genialer Schutzmechanismus, damit niemals fehlerhafter Code in Produktion geht.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

- `package`: Sind alle Tests grün, schnürt Maven das finale Paket – eine sogenannte „Fat JAR“. Diese einzelne Datei enthält Ihren gesamten Code, die statischen Frontend-Dateien und sogar den kompletten Webserver (*Tomcat*).
- `install` & `deploy`: Zum Schluss wird das fertige, geprüfte Paket entweder in das lokale Repository kopiert (`install`), damit andere Entwickler im Team es nutzen können, oder direkt auf einen echten Webserver bzw. in ein zentrales Firmen-Repository hochgeladen (`deploy`).

Das Fazit für die Praxis: In der täglichen Entwicklung nimmt uns die IDE die Arbeit ab und nutzt nur das absolute Minimum von Maven (`compile`). Erst der vollständige Maven-Lebenszyklus bis hin zu `package` oder `deploy` sorgt jedoch dafür, dass Ihr Projekt völlig unabhängig von IntelliJ, Betriebssystemen oder Cloud-Anbietern auf jedem Rechner der Welt exakt gleich getestet, gebaut und gestartet werden kann.

Session Tracking Vergleich

Session Tracking Vergleich

Vergleichen Sie die fünf gängigen Session-Tracking-Methoden. Füllen Sie dazu die folgende Tabelle aus:

Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	?	?	?
Hidden Fields	?	?	?
Cookies	?	?	?
HTTP Authorization Header	?	?	?
Session Tracking API (`HttpSession`)	?	?	?

Spring Boot REST-Controller



Spring Boot REST-Controller (1/2)

Implementieren Sie einen REST-Controller für ein einfaches Studierenden-Verwaltungssystem. Der Controller soll Anfragen annehmen und über ein bereitgestelltes Repository (`StudentRepository`) Daten abfragen oder verändern.

Das Projekt *studierendenverwaltung_vorlage* ist in Stud.IP hinterlegt. Dieses enthält bereits das Datenmodell `Student` und ein simuliertes `StudentRepository` (mit bereits hinterlegten Testdaten).

Spring Boot REST-Controller (2/2)

Erweitern Sie ausschließlich die vorgegebene Controller-Klasse `StudentController`, sodass die folgenden HTTP-Endpunkte unterstützt werden:

- `GET /students` — Gibt eine Liste aller Studierenden zurück (HTTP Status 200 OK).
- `GET /students/{id}` — Sucht einen Studierenden nach seiner ID. Wenn er existiert, wird er zurückgegeben (200 OK). Wenn nicht, soll der HTTP-Status 404 Not Found geliefert werden.
- `POST /students` — Legt einen neuen Studierenden an (201 Created). Der Endpunkt erwartet die Daten im JSON-Format im Request-Body. Reichen Sie im HTTP-Response-Header `Location` die URI des neuen Studierenden mit zurück (z. B. `/students/3`).
- `DELETE /students/{id}` — Löscht den Studierenden mit der angegebenen ID aus dem System (204 No Content).



MVC, MVP und MVVM im Vergleich

MVC, MVP und MVVM im Vergleich

1. Erklären Sie die Begriffe MVC, MVP und MVVM jeweils in ein bis zwei Sätzen.

2. Füllen Sie die folgende Tabelle aus:

Aspekt	MVC	MVP	MVVM
Wer kommuniziert mit dem Model?	?	?	?
Reaktion auf Änderungen	?	?	?
Testbarkeit	?	?	?
Typische Plattformen/Frameworks	?	?	?

3. Warum gilt das MVP-Muster als besonders testfreundlich?

Materialien zum Übungsblatt

- Aufgabe *Spring Boot REST Controller*: Vorlagendateien Studierendenverwaltung (`studierendenverwaltung_vorlage.zip`)